

ARTIFICIAL INTELLIGENCE

Assignment - 1

AI-Based Smart Parking System.

Submitted By

Sachin Biju
Group 6

S7 CSEA
Roll No:53

AI-Based Smart Parking System.

1 Introduction

The AI Smart Parking System is an intelligent agent that detects vacant parking slots and guides vehicles to the nearest available parking space. It uses AI, sensors, and cameras to monitor parking occupancy in real time. The system reduces the time spent searching for parking, minimizes traffic congestion, and improves parking space utilization. By providing accurate and timely guidance, it offers a smart, efficient, and user-friendly solution for modern parking management.

2 PEAS Representation

Agent Type	Performance Measure	Environment	Actuators	Sensors
AI-Based Smart Parking System	Minimize search time	Parking lot	Display parking guidance	Ultrasonic sensors
	Efficient slot utilization	Vehicles	Assign parking slot	CCTV cameras
	Fast slot allocation	Entry/exit gates	Control entry gate	Entry/exit sensors
	Accurate vehicle guidance	Parking slots	Send mobile alerts	Parking database

Table 1: PEAS Representation of the AI-Based Smart Parking System.

3 Environment Properties

Property	Type	Justification
Observability	Partially Observable	The agent observes parking slot occupancy through sensors and cameras but cannot predict future vehicle arrivals or unexpected sensor failures.
Determinism	Stochastic	Parking slot availability changes unpredictably as vehicles enter and leave the parking facility.
Dynamics	Dynamic	The environment continuously changes because parking slots become occupied or vacant in real time.
Discreteness	Discrete	Parking slots, vehicles, entry gates, and exits are finite and distinct entities.
Agents	Multi-Agent	Multiple vehicles interact with the parking system simultaneously while the intelligent agent allocates parking spaces.
Knowledge	Known	The parking layout, slot locations, and allocation rules are predefined and available to the intelligent agent.

Table 2: Environment Properties of the AI-Based Smart Parking System.

4 Rationality Measure

A rational agent selects the action that maximizes its performance measure based on the information available from sensors and the predefined parking rules. In the proposed AI-Based Smart Parking System,

the intelligent agent continuously monitors the occupancy status of parking slots using sensors and cameras. When a vehicle enters the parking facility, the intelligent agent identifies the nearest available parking slot using real-time occupancy data. It immediately reserves the selected slot and guides the driver through digital display boards or a mobile application. This ensures efficient parking allocation and prevents multiple vehicles from being assigned the same space. The system continuously updates parking availability as vehicles enter and leave the facility, enabling accurate real-time decisions. It consistently selects the nearest available parking space, reducing search time and traffic congestion. This improves parking space utilization and enhances the overall parking experience.

5 Innovation Point

The proposed system uses an intelligent AI agent to automatically detect vacant parking slots and guide vehicles to the nearest available space, improving parking efficiency and reducing search time. It can be further enhanced with features such as AI-based parking reservation, automatic license plate recognition (ANPR), and predictive parking availability to improve parking management and enhance the overall user experience.

6 Diagram

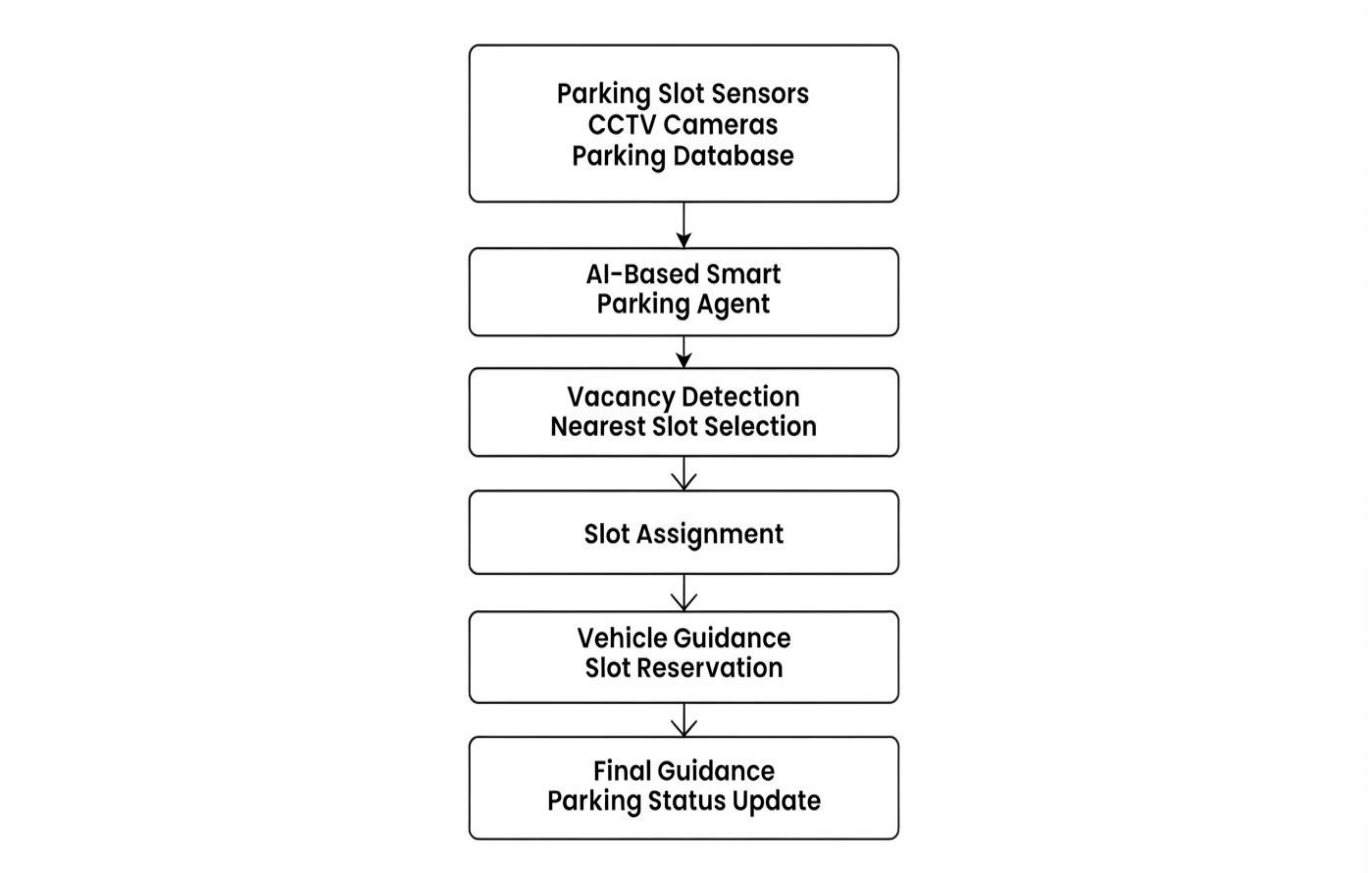


Figure 1: Architecture of the AI-Based Smart Parking System

7 Conclusion

The AI-Based Smart Parking System provides an efficient and intelligent solution for detecting vacant parking slots and guiding vehicles to the nearest available parking space. By continuously monitoring parking occupancy, selecting the nearest vacant slot, and providing real-time guidance, the system improves parking efficiency, maximizes space utilization, and reduces traffic congestion, fuel consumption, and driver search time.

8 Simulation

The simulation of the AI-Based Smart Parking System demonstrates the working of the intelligent agent in detecting vacant parking slots and guiding vehicles to the nearest available parking space. The system first collects real-time data from parking slot sensors, CCTV cameras, and the parking database to determine the occupancy status of each parking slot. The AI agent then analyzes the available parking spaces, identifies the nearest vacant slot, and reserves it for the arriving vehicle to avoid duplicate allocation.

After processing the parking request, the system guides the driver to the assigned parking space through digital display boards or a mobile application. It continuously updates parking availability as vehicles enter and leave the facility. The simulation demonstrates improved parking efficiency, reduced search time, and effective parking space utilization.

9 Screenshots of Full Code

The following screenshots show the complete implementation of the AI-Based Smart Parking System

```
1 import heapq
2 print("=" * 70)
3 print("      AI-BASED SMART PARKING SYSTEM")
4 print("=" * 70)
5
6 # -----
7 # INPUT: Parking Facility Layout (Multi-Level)
8 # -----
9 # Each slot: level, slot_id, distance_from_entrance (m), occupied status
10 parking_lot = [
11     {"level": 1, "slot": "A1", "distance": 5, "occupied": True},
12     {"level": 1, "slot": "A2", "distance": 8, "occupied": False},
13     {"level": 1, "slot": "A3", "distance": 3, "occupied": False},
14     {"level": 1, "slot": "A4", "distance": 12, "occupied": True},
15     {"level": 1, "slot": "A5", "distance": 2, "occupied": False},
16     {"level": 1, "slot": "A6", "distance": 15, "occupied": False},
17     {"level": 1, "slot": "A7", "distance": 7, "occupied": True},
18     {"level": 1, "slot": "A8", "distance": 4, "occupied": False},
19     {"level": 2, "slot": "B1", "distance": 18, "occupied": False},
20     {"level": 2, "slot": "B2", "distance": 28, "occupied": True},
21     {"level": 2, "slot": "B3", "distance": 16, "occupied": False},
22     {"level": 2, "slot": "B4", "distance": 22, "occupied": False},
23     {"level": 2, "slot": "B5", "distance": 19, "occupied": True},
24     {"level": 2, "slot": "B6", "distance": 17, "occupied": False},
25 ]
26
27 n_slots = len(parking_lot)
28 print(f"\nInitializing parking facility with {n_slots} slots across 2 levels...")
29
30 print("\n" + "-" * 70)
31 print("SENSOR SCAN: Current Slot Status")
32 print("-" * 70)
33 print("{:<8}{:<8}{:<15}{:<12}".format("Level", "Slot", "Distance(m)", "Status"))
34 print("-" * 70)
35 for s in parking_lot:
36     status = "Occupied" if s["occupied"] else "Vacant"
37     print("{:<8}{:<8}{:<15}{:<12}".format(s["level"], s["slot"], s["distance"], status))
38
```

Figure 2: Program Initialization and Sensor Scan Module

```

# -----
# VEHICLE ARRIVAL QUEUE
# -----
# Vehicles arriving at the entrance requesting a parking slot
vehicles = [
    {"vehicle_id": "KL07-AI-1234", "arrival_time": "09:01 AM"},
    {"vehicle_id": "KL07-BZ-5678", "arrival_time": "09:02 AM"},
    {"vehicle_id": "KL07-CX-9012", "arrival_time": "09:04 AM"},
]

print("\n" + "-" * 70)
print("VEHICLE ARRIVAL QUEUE")
print("-" * 70)
print("{:<18}{:<15}".format("Vehicle ID", "Arrival Time"))
print("-" * 70)
for v in vehicles:
    print("{:<18}{:<15}".format(v["vehicle_id"], v["arrival_time"]))

```

Figure 3: Vehicle Arrival QueueModule

```

57 # -----
58 # AI AGENT: Vacant Slot Detection using a Min-Heap
59 # -----
60 def build_vacancy_heap(lot):
61     """Builds a min-heap of all currently vacant slots ordered by
62     distance from the entrance, so the nearest slot can always be
63     retrieved in O(log n) time."""
64     heap = []
65     for s in lot:
66         if not s["occupied"]:
67             heapq.heappush(heap, (s["distance"], s["level"], s["slot"]))
68     return heap
69
70
71 def guide_vehicle_to_nearest_slot(lot, vehicle):
72     """AI decision procedure: picks the nearest vacant slot for a
73     single vehicle, reserves it, and returns the guidance details."""
74     heap = build_vacancy_heap(lot)
75
76     if not heap:
77         return None
78
79     distance, level, slot_id = heapq.heappop(heap)
80
81     # Reserve the slot so the next vehicle cannot be guided to it
82     for s in lot:
83         if s["slot"] == slot_id:
84             s["occupied"] = True
85             break
86
87     return {
88         "vehicle_id": vehicle["vehicle_id"],
89         "slot": slot_id,
90         "level": level,
91         "distance": distance,

```

Figure 4: AI-Based Nearest Slot Allocation Algorithm

```

92     }
93
94
95 print("\n" + "=" * 70)
96 print("AI AGENT: Detecting vacancies and guiding each vehicle...")
97 print("=" * 70)
98
99 guidance_log = []
100 for vehicle in vehicles:
101     result = guide_vehicle_to_nearest_slot(parking_lot, vehicle)
102     if result:
103         guidance_log.append(result)
104         print(f"\nVehicle {result['vehicle_id']}")
105         print(f"    Assigned Slot    : {result['slot']} (Level {result['level']})")
106         print(f"    Distance          : {result['distance']} m from entrance")
107         print(f"    Guidance          : Proceed to Level {result['level']}, "
108               f"slot {result['slot']} is {result['distance']}m ahead.")
109     else:
110         print(f"\nVehicle {vehicle['vehicle_id']}: No vacant slot available. "
111               f"Facility is FULL.")
112

```

Figure 5: AI-Guided Vehicle Allocation Module

```

113 # -----
114 # PERFORMANCE EVALUATION
115 # -----
116 total_slots = len(parking_lot)
117 occupied_slots = sum(1 for s in parking_lot if s["occupied"])
118 vacant_slots = total_slots - occupied_slots
119 occupancy_rate = (occupied_slots / total_slots) * 100
120 avg_guidance_distance = (
121     sum(g["distance"] for g in guidance_log) / len(guidance_log)
122     if guidance_log else 0
123 )
124
125 print("\n" + "=" * 70)
126 print("AI PERFORMANCE REPORT")
127 print("=" * 70)
128 print("Total Slots           :", total_slots)
129 print("Occupied Slots          :", occupied_slots)
130 print("Vacant Slots             :", vacant_slots)
131 print("Occupancy Rate           : {:.1f}%".format(occupancy_rate))
132 print("Vehicles Guided          :", len(guidance_log))
133 print("Avg. Guidance Distance   : {:.1f} m".format(avg_guidance_distance))
134 print("Guidance Time per Vehicle: < 1 sec")
135 print("Status                   : Optimal")
136
137 print("\nAll vehicles processed and slot allocation completed successfully.")

```

Figure 6: Performance Evaluation and Final Output

10 Output

The following figures show the execution of the program and the parking guidance generated by the intelligent agent. They display the detection of vacant parking slots, vehicle allocation, and the final parking status report produced by the AI-Based Smart Parking System.

=====AI			
-BASED SMART PARKING SYSTEM			
=====			
Initializing parking facility with 14 slots across 2 levels...			

SENSORSCAN: Current Slot Status			
Level	Slot	Distance (m)	Status
1	A1	5	Occupied
1	A2	8	Vacant
1	A3	3	Vacant
1	A4	12	Occupied
1	A5	2	Vacant
1	A6	15	Vacant
1	A7	7	Occupied
1	A8	4	Vacant
2	B1	18	Vacant
2	B2	20	Occupied
2	B3	16	Vacant
2	B4	22	Vacant
2	B5	19	Occupied
2	B6	17	Vacant

VEHICLE	ARRIVAL	QUEUE	
Vehicle	ID	ArrivalTime	

KL07-AI-1234		09:01AM	
KL07-BZ-5678		09:02AM	
KL07-CX-9012		09:04AM	

Figure 7: Sensor Scan Output and Vehicle ArrivalQueue

```

=====AI
PERFORMANCE REPORT
=====
TotalSlots          :14
OccupiedSlots       :8
VacantSlots         :6
OccupancyRate       :57.1%
Vehicles Guided     : 3 Avg.
Guidance Distance:  3 . 0 m
Guidance Time per Vehicle: <
lsecStatus          : Optimal

Allvehiclesprocessedandslotallocationcompletedsuccessfully.

```

Figure 8: Final AI PerformanceReport